

Collision-Free Video Synopsis Incorporating Object Speed and Size Changes

Yongwei Nie, *Member, IEEE*, Zhenkai Li, Zhensong Zhang, Qing Zhang, Tiezheng Ma, Hanqiu Sun

Abstract—This paper presents a new surveillance video synopsis method which performs much better than previous approaches in terms of both compression ratio and artifact. Previously, a surveillance video was usually compressed by shifting the moving objects of that video forward along the time axis, which inevitably yielded serious collision and chronological disorder artifacts between the shifted objects. The main observation of this paper is that these artifacts can be alleviated by changing the speed or size of the objects, since with varied speed and size the objects can move more flexibly to avoid collision points or to keep chronological relationships. Based on this observation, we propose a video synopsis method that performs object shifting, speed changing, and size scaling simultaneously. We show how to integrate the three heterogeneous operations into a single optimization framework and achieve high-quality synopsis results. Unlike previous approaches that usually use alternative optimization strategies to solve synopsis optimizations, we develop a Metropolis sampling algorithm to find the solution for our three-variable optimization problem. A variety of experiments demonstrate the effectiveness of our method.

Index Terms—Surveillance video synopsis, collision avoiding, speed/size change, Metropolis sampling.

I. INTRODUCTION

NOWADAYS, surveillance cameras are all around the world, capturing tremendous raw video materials every day. It is difficult to directly browse, store, or transfer these raw videos, as they are usually 24-hours long and contain limited useful information. Therefore, video synopsis, the technique that generates a short representation for a redundant video, has obtained lots of research during the last several decades.

Surveillance videos are special in that they usually have static backgrounds. The moving objects in these videos can be detected and extracted relatively easily. Therefore, an effective algorithm to compress a surveillance video is to directly shift the moving objects in the video forward along the time axis [1]–[3]. With the shifting operation, we can show activities originally occurring at different times simultaneously and eliminate the redundancies between the objects themselves. This is useful in saving most of the object activities and achieving a high compression ratio, but it also brings two problems. Firstly, the shifted objects may collide with each other. Secondly, the chronological orders between objects may

be destroyed. When the input video is compressed more, the two problems will become more severe.

Several approaches [4]–[6] have been developed to solve the collision and chronological disorder problems. Besides shifting objects along the time axis, Nie et al. [4] also shifted them along the x and y axes. Due to the use of extra spatial space, the method can achieve a high compression ratio while eliminating collision and time disorder artifacts. However, it also destroys spatial position relationships between objects and has to synthesize a new background that matches with the spatially shifted objects. Li et al. [5] found that the collisions between two objects can be avoided by resizing the objects down. This idea is straightforward and effective in avoiding collision artifacts. But the objects can not be zoomed to be very small, otherwise they will become too tiny to be seen. Therefore, the compression ratio is still limited. Recently, He et al. [6] has proposed to construct a potential collision graph based on which the collision mitigating problem is formulated as a graph coloring problem. In their work, the objects themselves are kept intact. In this situation, the avoiding of the collision artifacts is at the expense of the increase of the compression ratio.

In this paper, we propose a novel surveillance video synopsis method, which performs much better than previous approaches in terms of both compression ratio and artifact. The basic idea is to change the moving objects themselves rather than keeping them intact during the process of shifting. Specifically, in addition to changing the size of objects like Li et al. [5], we further change the moving speed of the objects adaptively. We find that allowing the speed and size to be changeable, especially the changing of speed, makes the objects move more flexibly to avoid collision and chronological disorder artifacts. Moreover, since increasing the moving speed of objects means shortening the time duration of them, a shorter synopsis video can even contain more speed-increased objects, and accordingly a higher compression ratio can be achieved.

Based on the above analysis, we propose a synopsis method that performs object shifting, speed changing, and size scaling simultaneously. The object shifting is used to compact moving objects of the input video into a short time space, while the speed changing and size scaling are used to change the objects themselves to increase the flexibility of the shifting. However, the challenge is how to integrate the three heterogeneous operations into a single optimization framework. We need to construct a unified energy function involving shift, speed, and size variables. The introducing of the speed variables makes the computation of the energy function very complicated since

Yongwei Nie, Zhenkai Li, and Tiezheng Ma are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China, 510006. Yongwei Nie's E-mail is nieyongwei@scut.edu.cn

Zhensong Zhang and Hanqiu Sun are with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Shatin, New Territories, Hong Kong.

Qing Zhang is with the School of Data and Computer Science, Sun Yat-Sen University, Guangzhou, China.

any local change to the speed of an object will cause a global change to the whole form of the object over all frames, then affecting the interactions between objects. In addition, since among the three variables there exist both continuous and discrete variables, how to solve the unified optimization built on them is also a problem.

In order to solve the above problems, we extract object tubes where a tube represents all occurrences of an object over video frames, divide the tubes into segments, and change the speed and size of each segment adaptively. For convenience, we propose a rendering mechanism that can render a synopsisized object given its shift, speed, and size variables. Then with the rendering mechanism, we define our unified optimization that is composed of five energy terms which are used to preserve object activities, to avoid collision and chronological disorder artifacts, and to make the speed and size variables change smoothly in limited ranges. Finally, instead of solving the unified formulation by an alternative optimization strategy that usually adopted by previous works like [4], [5], we propose a Metropolis sampling algorithm that is more effective to find the solution for our problem.

In summary, the contribution of this paper is in the idea of introducing object speed changing into surveillance video synopsis, in the formulation that optimizes object shifting, speed changing, and size scaling simultaneously, together with the proposing of the Metropolis sampling algorithm used to solve the optimization. A variety of experiments demonstrate that the proposed method can produce impressive synopsis results that are better than previous approaches in terms of both compression ratio and artifact.

II. RELATED WORK

Existing video synopsis approaches can be roughly classified into frame-based video abstraction and object-based video synopsis.

A. Frame-based video abstraction

Frame-based approaches treat video frames as building blocks. According to whether they are based on video frames or shots, we can classify them as key-frame based approaches or video skimming approaches (see survey [7]). Key-frame based approaches usually present some criteria that can determine the importance of a video frame [8]–[11]. For example, Liu et al. [8] proposed a novel criterion called shot reconstruction degree for key frame selection based on the degree of retaining motion dynamics of a video shot. Zhang et al. [10] extracted key frames by identifying motion stage changes on a spatiotemporal slice. Kumar et al. [11] employed K-means clustering to extract key frames quickly and determined the number of the clusters based on Davies-Bouldin Index. In the works of [12]–[14], auxiliary data sources such as speech and eye tracker data were also taken into consideration when designing the criteria. Two interesting topics related to key-frame based abstraction are fast-forwarding [15]–[17] and time-lapse [18], [19]. Both of them optimize the sampling rate when playing a video. In comparison, video skimming approaches [20]–[24] try to divide videos into shots, and select important shots instead of individual frames. For example,

Molino et al. [22] introduced an interactive video summarization approach that enquires and accepts a video shot based on the current and previous questions and the corresponding answers. Li et al. [23] proposed a general video summarization framework for both raw and edited videos. Generally speaking, frame/shot-based approaches are robust and can be applied to any type of videos, e.g., from news to sports, from movies to any web videos, etc. However, for specific videos, such as those captured by surveillance cameras, they cannot achieve satisfactory compression ratio.

B. Object-based Video Synopsis

It has been demonstrated that putting video content that originally occur at different times together and showing them simultaneously is an effective video editing and summarizing tool [25]–[31]. For dynamic videos, spatiotemporal video portions are usually extracted, then packed and stitched together into a small output video [25], a multi-level storyline [28], or a multi-scale tapestry [27]. However, there usually exist apparent seams between adjacent portions being stitched together. On the other hand, for videos with static background, approaches like [29]–[31] directly manipulate objects along the time-line. In this way, they can focus on visual content rather than time when browsing a video [29], [30], or produce visual special effects such as slow motion effects, fast motion effects, etc., [31]. Rav et al. [1] and Pritch et al. [2], [3] are the first to summarize surveillance videos by shifting objects along the time axis. They tracked and extracted the same object in different frames and viewed them as a whole object tube. Then by shifting tubes forward along the time axis, the temporal redundancies among the tubes were eliminated effectively. Many approaches follow the pioneering work [3] and improve it from various of aspects. For example, approaches of [32]–[35] summarize only abnormal or action-similar objects into the synopsis video. Works of [36], [37] extract more fluent tubes, or cope with tubes in complex scenes. Since the optimization in [3] is time-consuming, approaches of [38]–[41] try to accelerate it and make it work online. In [42]–[44], the input video is condensed by ribbon carving. Nie et al. [45] extended the idea of [3] for part-based object action synopsis. Another hot topic is to eliminate collision artifacts that may occur between shifted tubes. For this, Nie et al. [4] shifted tubes in both the spatial and temporal space. Li et al. [5] scaled down objects while shifting them. Recently, He et al. [6] reformulated video synopsis as a graph coloring problem in which potential collision nodes are colored with different colors.

III. PROPOSED METHODOLOGY

A. Overview

The main idea of this paper is expressed in Fig. 1. The left shows object shifting, the middle shows object resizing, and the right shows object speed changing. The contribution of this paper is integrating the three components together into a single optimization framework, achieving higher compression ratio when compressing surveillance videos.

In the left of Fig. 1, the color curves represent object tubes of a video. The object shifting component shifts the tubes

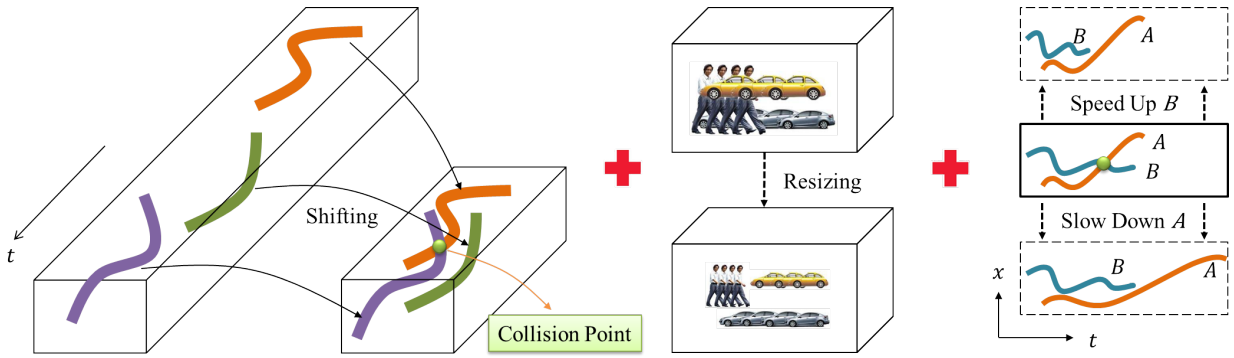


Fig. 1. The overview of our algorithm. Please refer to Sec. III-A for more descriptions.

along the time axis, compacting them into a shorter temporal space, which however causes collisions between the tubes. The middle of Fig. 1 (borrowed from [5]) shows that the collisions between objects can be eliminated by scaling down the objects. On the right side of Fig. 1, we show the collisions can also be removed by changing the speed of the objects. For example, objects A and B collide with each other originally in the middle. We speed up B to shorten its trajectory (the upper one), or slow down A to stretch its trajectory (the bottom one), both making the two objects miss the collision point.

In this paper, the three components are not executed separately. Instead, we formulate them into a unified optimization, and then use Metropolis algorithm to find the solution.

B. Background Estimation and Object Tube Extraction

The first step is to extract object tubes. Following [3]–[5], we estimate static background first, then subtract the background from video frames to detect moving objects.

Like [3]–[5], we estimate the background of a surveillance video by applying a temporal median filter to the video. Considering the illumination may change from time to time, we compute a background for every minute of the video. The value of each pixel of a background is the median one among all the pixels at the same location but across all the frames of the one minute shot. Fig. 2 (b) shows a background estimated.

Next, we extract object tubes. An object tube is a set of continuous occurrences of the same object over video frames. Fig. 2 (c) gives an example. To extract each occurrence, we simply subtract the original frame by the corresponding background image. Pixels with subtraction difference larger than 10 are viewed as foreground pixels. Then, morphological operations including dilation, erosion, etc., are used to compute the object mask as shown in the top left corner of Fig. 2 (c). If two objects collide with each other in a frame, their masks will be mixed together and cannot be separated. In this situation, we view them as a single object. Finally, we connect occurrences of different frames together to form an object tube if they are overlapped with each other one by one along the time.

The above background and object tube extraction strategy works well for the experiments in this paper. However, it may fail when the videos are captured in more complex illumination conditions. In these cases, Gaussian Mixed Model

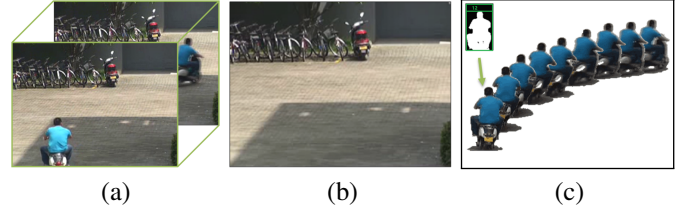


Fig. 2. (a) An input surveillance video. (b) The estimated background. (c) The tube of an object.

based foreground extraction methods [46] and recent object tracking methods [47] can be used to solve the problems.

C. Notations and Rendering Mechanism

In this section, we introduce the notations that will be used in the definition of our video synopsis formulation. We also propose a rendering mechanism that renders a target object tube in the synopsis video given the source tube and the corresponding shift, size, and speed variables.

Let N be the number of tubes of an input video. As shown in Fig. 3 (a), we use O_i to denote the i^{th} object tube (or object), and we use them interchangeably in the following), L_i be the length of O_i , and f_i be the index of the frame where the object O_i starts to occur in the original video. The task of object shifting is to find for each object O_i a shifting offset s_i such that $\tilde{f}_i = f_i + s_i$, where $\tilde{f}_i$ is the first frame where the target tube $\tilde{O}_i$ starts to appear in the synopsis video, as shown in Fig. 3 (b). Besides being shifted, the object is also resized and/or the speed of it is changed. For example, the purple and red curves become thinner from Fig. 3 (a) to (b), illustrating that they are scaled down. The gold tube becomes longer from (a) to (b), illustrating that it is slowed down. To resize an object or change the speed of it, we first divide an object tube O_i into a set of segments as shown in (c), and let M_i be the number of the segments. Then, the speed or size of O_i is changed by changing the speed or size of each segment j according to speed scaling factor v_i^j or size scaling factor r_i^j of the segment.

About the speed changing variables $\{v_i^j\}_{i \in [1, N], j \in [1, M_i]}$ and size changing variables $\{r_i^j\}_{i \in [1, N], j \in [1, M_i]}$, three design considerations are worthy noted. First, there are two ways to divide a tube into segments. We can divide all the tubes into

the same number of segments, or we can divide every object tube into segments of the same length. We choose the later one, and denote the length of each segment by ℓ which is an important hyperparameter that can be adjusted by user. The role of ℓ will be explained later in the experimental section. Second, different from [5], we can no longer set a resizing variable for each frame of an original tube, as the length of the tube may be changed due to speed changes. Therefore like speed variables, the resizing variable is also attached to each segment of a tube the number of which does not change from O_i to $\tilde{O}_i$. But in this way the frames of the same segment have to undergo the same resizing factor. Third, we confine the speed or size variables in a limited range: $v_i^j \in [a, b]$ and $r_i^j \in [c, d]$ for any i and j . For a and b , they should satisfy $0 < a \leq 1 \leq b \leq \ell$. The upper bound of b is ℓ in which case only one frame of a segment of length ℓ is sampled and preserved into the synopsis video. In this paper, $[a, b]$ is set as $[0.3, 2]$, while $[c, d]$ is set as $[0.5, 1]$. That means an object's speed can be changed to at most $2\times$ and at least $0.3\times$ the original speed. And the size of an object can be reduced to at most the half of it. We do not enlarge the size of an object in this paper.

Now we introduce our rendering mechanism that renders tube $\tilde{O}_i$ given O_i and the accompanying shift variable s_i , speed variables $\{v_i^j\}_{j \in [1, M_i]}$, and size variables $\{r_i^j\}_{j \in [1, M_i]}$. To render $\tilde{O}_i$, we should know: (a) the temporal position (i.e., the frame index) of each occurrence of $\tilde{O}_i$ in the synopsis video, and (b) the occurrences themselves.

Firstly, we use the following way to change the j^{th} segment of O_i by speed scaling factor v_i^j . The original length of the segment is ℓ . The target length becomes $\ell_i^j = \lfloor \frac{\ell}{v_i^j} \rfloor$, where $\lfloor \cdot \rfloor$ is the round operator to find the nearest integer. The total length of $\tilde{O}_i$ is computed as the sum of all its segments: $\tilde{L}_i = \sum_j \ell_i^j$. Now we know the temporal position of the k^{th} occurrence of $\tilde{O}_i$ in the synopsis video, which is $\tilde{f}_i + k$, where $k \in [0, \tilde{L}_i)$. Recall that a source segment of length ℓ becomes a target segment of length ℓ_i^j . Then each occurrence k' of the target segment is a copy of the occurrence $\lfloor \frac{\ell-1}{\ell_i^j-1} \cdot k' \rfloor$ of the source segment, where $k' \in [0, \ell_i^j)$. The copy is then resized by r_i^j , and finally rendered into the synopsis video according to the temporal position of it.

For simplicity, we use a function Φ to summarize the above rendering mechanism between O_i and $\tilde{O}_i$, but in the reverse way:

$$O_i(k^*) = \Phi(\tilde{O}_i(k)). \quad (1)$$

where we use $\tilde{O}_i(k)$ to denote the k^{th} occurrence of $\tilde{O}_i$. By Φ , we find its original occurrence k^* of tube O_i .

D. Unified Object Shifting, Resizing, and Speed Changing

In this section, we describe our unified video synopsis formulation. Given the above notations, the unified optimization framework aims at finding the best values for object shifting variables $\mathcal{F} = \{\tilde{f}_i\}_{i \in [1, N]}$ (we directly optimize $\tilde{f}_i$ instead of s_i), speed scaling variables $\mathcal{V} = \{v_i^j\}_{i \in [1, N], j \in [1, M_i]}$, and

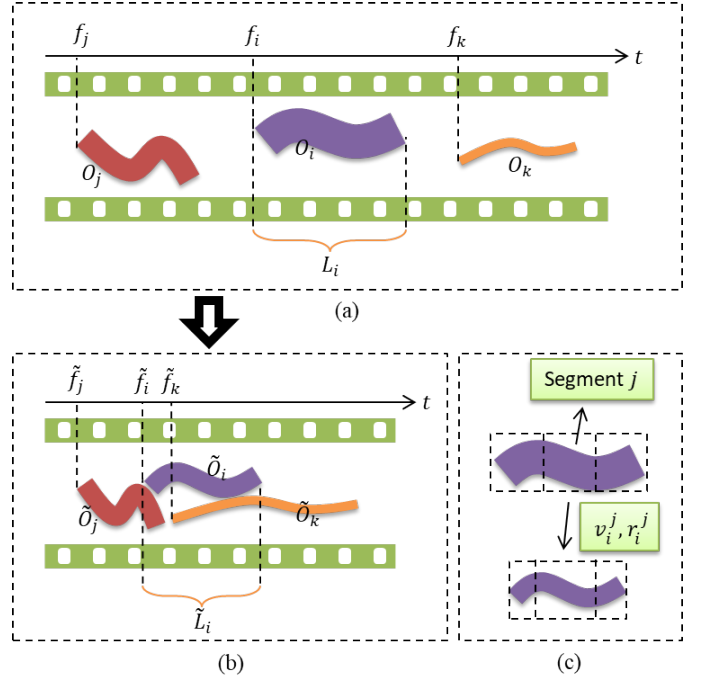


Fig. 3. Notations used in this paper. Please refer to Sec. III-C for more information.

resizing variables $\mathcal{R} = \{r_i^j\}_{i \in [1, N], j \in [1, M_i]}$. The best values can be obtained by optimizing the following energy function:

$$E(\mathcal{F}, \mathcal{V}, \mathcal{R}) = E_a(\mathcal{F}, \mathcal{V}) + \omega_c E_c(\mathcal{F}, \mathcal{V}, \mathcal{R}) + \omega_t E_t(\mathcal{F}) + \omega_s E_s(\mathcal{V}, \mathcal{R}) + \omega_d E_d(\mathcal{V}, \mathcal{R}), \quad (2)$$

where E_a , E_c and E_t are terms encoding the activity, collision, and chronological losses, E_s is a term making the speed and size variables vary smoothly over segments, and E_d is a term preventing large changes of speed and size. Finally, ω_* are weights to balance these terms. Note that the weight of term E_a is always 1.

1) *Activity Term E_a* : This term is used to preserve the objects into the synopsis video. In [3], Pritch et al. computed the term as the activities shifted out of the synopsis video. We follow this idea, but change it to the following form that considers the factors of object speed and size changes:

$$E_a(\mathcal{F}, \mathcal{V}) = \sum_i \sum_{k=0}^{\tilde{L}_i-1} \delta(\tilde{f}_i + k < 0 | \tilde{f}_i + k \geq L_s) \cdot \Psi(\Phi(\tilde{O}_i(k))), \quad (3)$$

where $\delta(\cdot)$ is a function which outputs 1 if its argument is true and 0 otherwise. k indexes the occurrence of a tube. If $\tilde{f}_i + k < 0$ or $\tilde{f}_i + k \geq L_s$, the occurrence is moved out of the synopsis. Its activity is then added to the term. The activity of the k^{th} occurrence of $\tilde{O}_i$ is represented by $\Psi(\Phi(\tilde{O}_i(k)))$, which is the sum of differences between the pixels of the source occurrence and the corresponding background pixels. Note that this term is only related to variables $\mathcal{F}$ and $\mathcal{V}$.

2) *Collision Term E_c* : The collision term is defined on all pairs of tubes $\tilde{O}_{i_1}$ and $\tilde{O}_{i_2}$. To compute the collision energy, we first find the time interval during which both the two tubes occur, and then sum up the intersection area of the occurrences

during the time interval. Let $k_1 = \max\{\tilde{f}_{i_1}, \tilde{f}_{i_2}\}$, and $k_2 = \min\{\tilde{f}_{i_1} + \tilde{L}_{i_1}, \tilde{f}_{i_2} + \tilde{L}_{i_2}\}$, then the shared time interval by $\tilde{O}_{i_1}$ and $\tilde{O}_{i_2}$ is from k_1 to k_2 . The collision term is then defined as:

$$E_c(\mathcal{F}, \mathcal{V}, \mathcal{R}) = \sum_{i_1, i_2} \sum_{k=k_1}^{k_2} \frac{\text{Rect}(\tilde{O}_{i_1}(k - \tilde{f}_{i_1})) \cap \text{Rect}(\tilde{O}_{i_2}(k - \tilde{f}_{i_2}))}{r(k - \tilde{f}_{i_1}) \cdot r(k - \tilde{f}_{i_2})}. \quad (4)$$

Now k indexes a frame of the synopsis video. Subtracting $\tilde{f}_{i_1}$ and $\tilde{f}_{i_2}$ from k gives the occurrence indices of $\tilde{O}_{i_1}$ and $\tilde{O}_{i_2}$. The $\text{Rect}(\cdot)$ operator outputs the axis-aligned minimum bounding box that encloses the occurrence input to it. The symbol $\cap$ computes the intersection area between the two bounding boxes. $r(k - \tilde{f}_{i_1})$ and $r(k - \tilde{f}_{i_2})$ are resizing factors of the two occurrences. It is worth noting that the division by the resizing factors is necessary. Otherwise the optimization will prefer to scale down two objects even though they still collide with each other after the scaling.

3) *Chronological Term E_t* : A tube appears earlier in the original video should be earlier in the synopsis video too. The chronological term is used to penalize the invert of the chronological order of any pair of object tubes. To determine whether a chronological order being inverted or not, we first calculate $(f_{i_1} - f_{i_2}) \cdot (\tilde{f}_{i_1} - \tilde{f}_{i_2})$. If it is positive, the chronological order is preserved, otherwise inverted. Let $k_1 = \min\{\tilde{f}_{i_1}, \tilde{f}_{i_2}\}$, and $k_2 = \max\{\tilde{f}_{i_1}, \tilde{f}_{i_2}\}$. And let $i = i_1$ if $\tilde{f}_{i_1} \leq \tilde{f}_{i_2}$, otherwise $i = i_2$. We define the chronological term as:

$$E_t(\mathcal{F}) = \sum_{i_1, i_2} \delta((f_{i_1} - f_{i_2}) \cdot (\tilde{f}_{i_1} - \tilde{f}_{i_2}) < 0) \cdot \sum_{k=k_1}^{k_2} \Psi(\Phi(\tilde{O}_i(k - \tilde{f}_i))), \quad (5)$$

i.e., we just penalize the disordered activities as shown in the shaded area of Fig. 4 (a). However, the term defined in the above equation fails to work as shown in the cases of Fig. 4 (b). In the two cases, the penalized activities are equal, but the degree of the violation of the chronological order are actually very different. To overcome this problem, we argue that the chronological term should be proportional to the temporal distance between two shifted tubes. Therefore we define the following weight:

$$\omega(\tilde{f}_{i_1}, \tilde{f}_{i_2}) = \frac{1}{\left(1 - \frac{\min\{\|\tilde{f}_{i_1} - \tilde{f}_{i_2}\|, \xi\}}{\xi}\right)^5} - 1, \quad (6)$$

where $\|\tilde{f}_{i_1} - \tilde{f}_{i_2}\|$ is the temporal distance, ξ is the maximum number of frames that we can tolerate to be inverted, and ϵ is a tiny number 10^{-9} to prevent division by zero. The weight increases significantly as the temporal distance increases, and the increasing rate is determined by ξ that is automatically determined as follows. Assuming $f_1, f_2, \dots, f_N$ are ascending numbers. We compute $f_3 - f_1, f_4 - f_2, \dots, f_N - f_{N-2}$ first,

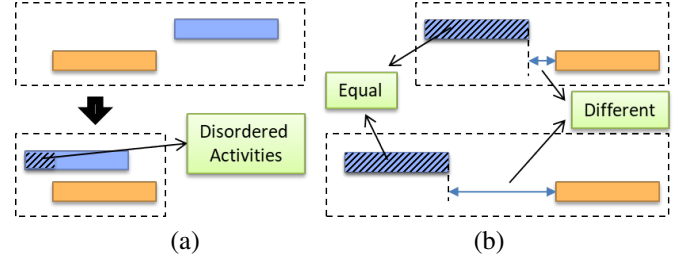


Fig. 4. (a) The shaded region illustrates the disordered activities. (b) In both cases, the disordered activities are equal, but the degree of the violation of the chronological order are different.

then the median of these numbers is selected as ξ . Our final chronological term is then refined by adding the weight term:

$$E_t(\mathcal{F}) = \sum_{i_1, i_2} \delta((f_{i_1} - f_{i_2}) \cdot (\tilde{f}_{i_1} - \tilde{f}_{i_2}) < 0) \cdot \omega(\tilde{f}_{i_1}, \tilde{f}_{i_2}) \cdot \sum_{k=k_1}^{k_2} \Psi(\Phi(\tilde{O}_i(k - \tilde{f}_i))). \quad (7)$$

4) *Smooth Terms E_s* : We demand the speed and size variables of the same tube to vary smoothly over the segments of the tube. This is achieved by minimizing the second-order differences of these variables:

$$E_s = \sum_i \sum_{j=2}^{M_i-1} \left(2v_i^j - v_i^{j-1} - v_i^{j+1}\right)^2 + \left(2r_i^j - r_i^{j-1} - r_i^{j+1}\right)^2. \quad (8)$$

5) *Deviation Term E_d* : The speed and size should not be changed too much. For example, if an object is accelerated too much or zoomed to be too small, the user may not watch the activities of the object clearly. Therefore one one hand, we confine the speed and size scaling variables in $[a, b]$, and $[c, d]$ respectively, where $a = 0.3$, $b = 2$, $c = 0.5$, $d = 1$ as stated earlier. On the other hand, we penalize the changing of the speed and size variables by the following term:

$$E_d = \sum_i \frac{1}{M_i} \sum_{j=1}^{M_i} \exp\left(\frac{\sigma}{\min\left\{v_i^j, \frac{1}{v_i^j}\right\}}\right) + \exp\left(\frac{\sigma}{r_i^j}\right), \quad (9)$$

where σ is empirically set as 2, and the $\exp(\cdot)$ function significantly increases when the speed or size variable input to it deviates more from 1.

E. Minimizing by MCMC sampling

The energy function defined in Eq. 2 is highly multimodal, and is difficult to solve by exact optimization techniques. One idea is to find a local minimum solution by alternative optimization strategy, i.e., optimizing shift variables when fixing speed and size variables, and then optimizing speed variables when fixing shift and size variables, and so on. Such alternative optimization strategy is often used in previous approaches such as [4] and [5]. However, the method is not appropriate for our situation due to two reasons. First, while the shifting problem can be solved by Graph Cuts [48], and the resizing problem can be solved by any gradient descent algorithm [49], the speed changing is actually a very complex discrete

optimization problem. That is because although theoretically we can set v_i^j as any real number within $[a, b]$, the values that really work are those make $\frac{\ell}{v_i^j}$ be integer, as we cannot sample 0.3 frame or any kind of this. Second, even we solve the discrete problem with great effort, the solution by the alternative strategy is just a local minima. In this paper, we give up the alternative strategy, and propose to use a Markov chain Monte Carlo (MCMC) algorithm [50] to solve the video synopsis problem.

MCMC methods are used to generate samples from a distribution by constructing a reversible Markov-chain that has its equilibrium distribution as the distribution. To apply MCMC algorithm, we first transform our energy function of Eq. 2 to a Boltzmann-like density function:

$$p(\mathcal{F}, \mathcal{V}, \mathcal{R}) = \frac{1}{Z} \exp(-\beta E(\mathcal{F}, \mathcal{V}, \mathcal{R})), \quad (10)$$

where β is a temperature constant, and Z is the partition function that normalizes the distribution. Usually, the computing of Z is intractable. MCMC methods such as Metropolis-Hastings algorithm [51], [52] can explore the density function without computing Z . The Metropolis-Hastings algorithm begins with any initial sample, i.e., a configuration of $\mathcal{I} = (\mathcal{F}, \mathcal{V}, \mathcal{R})$. With the current configuration $\mathcal{I}$, it makes some modification to $\mathcal{I}$ to obtain a new configuration $\mathcal{I}^*$ by a proposal distribution $q(\mathcal{I}^*|\mathcal{I})$. The probability to accept the modification is:

$$\alpha(\mathcal{I} \rightarrow \mathcal{I}^*) = \min \left(1, \frac{p(\mathcal{I}^*)q(\mathcal{I}|\mathcal{I}^*)}{p(\mathcal{I})q(\mathcal{I}^*|\mathcal{I})} \right). \quad (11)$$

If the modification is accepted, the current configuration becomes $\mathcal{I}^*$, otherwise it remains as $\mathcal{I}$. The above process is iterated. After a number of steps of burning-in, the sampling conforms to the distribution of Eq. 10. We choose among all the sampled configurations with the highest probability according to Eq. 10 as the solution of Eq. 2.

The remaining problem is how to design the proposal distribution $q(\mathcal{I}^*|\mathcal{I})$ which should make both local proposals that make minor modifications to the configuration and global modifications that lead to significant rearrangements. Our proposal distribution chooses among the following four possible proposals with equal probability:

- Make a modification to some shift variable $\tilde{f}_i$ by randomly choosing an integer in the range $[0, L_s)$, where L_s is the length of the synopsis video.
- Randomly choose an integer e in the range $[\lfloor \frac{\ell}{b} \rfloor, \lfloor \frac{\ell}{a} \rfloor]$, then choose a speed variable v_i^j , and set it as $\frac{\ell}{e}$. We explain more about this proposal. For v_i^j , we cannot directly sample a real value in the range $[a, b]$, as the sampled values may bring no change. For example, let $\ell = 10$, and $v_i^j = 0.5$ or 0.51 . Both v_i^j produce a new segment of length 20 (i.e., $\lfloor 10/0.5 \rfloor = \lfloor 10/0.51 \rfloor = 20$). Therefore, we sample a new length between $\lfloor \frac{\ell}{b} \rfloor$ and $\lfloor \frac{\ell}{a} \rfloor$, and then compute a value v_i^j in turn.
- Perturb some resizing variable r_i^j by normal distribution $\mathcal{N}(0, 0.01^2)$. If the modified variable is out of the user defined range $[c, d]$, sample again.
- Randomly choose two tubes, and swap the values of their shift variables.

Algorithm 1 Metropolis Video Synopsis Algorithm.

Input: The number of iterations n_iters .

Output: A good solution of $\mathcal{I}$.

```

1: Initialize  $\mathcal{I}_0$ :  $\mathcal{F}_0$  is initialized by proportionally mapping
   tubes from the input video to the synopsis video;  $\mathcal{V}_0$  and
    $\mathcal{R}_0$  are initialized by random sampling within the user
   defined ranges.
2: for  $n = 1$  to  $n\_iters$  do
3:   switch RANDOMPROPOSAL()
4:   case shift
5:     Sample integer  $i \sim [1, N]$ 
6:     Generate  $\mathcal{I}'_n$  by sampling  $f'_i \sim [0, L_s)$ 
7:   case speed
8:     Sample integer  $i \sim [1, N]$ ,  $j \sim [1, M_i]$ 
9:     and  $e \sim [\lfloor \frac{\ell}{b} \rfloor, \lfloor \frac{\ell}{a} \rfloor]$ 
10:    Generate  $\mathcal{I}'_n$  by  $v_i^j \leftarrow \frac{\ell}{e}$ 
11:   case size
12:     Sample integer  $i \sim [1, N]$ ,  $j \sim [1, M_i]$ 
13:     Sample  $x \sim \mathcal{N}(0, 0.01^2)$ 
14:      $r_i^j \leftarrow r_i^j + x$ 
15:     if  $r_i^j$  not in  $[c, d]$ , go to line 13
16:     Generate  $\mathcal{I}'_n$  by new  $r_i^j$ 
17:   case swap
18:     Sample integer  $i_1 \sim [1, N]$ ,  $i_2 \sim [1, N]$ 
19:     Generate  $\mathcal{I}'_n$  by  $\tilde{f}_{i_1} \leftarrow \tilde{f}_{i_2}$  and  $\tilde{f}_{i_2} \leftarrow \tilde{f}_{i_1}$ 
20:    $\alpha \leftarrow \min \left( 1, \frac{p(\mathcal{I}'_n)}{p(\mathcal{I}_{n-1})} \right)$ 
21:   Sample  $t \sim [0, 1]$ 
22:   if  $t < \alpha$  then  $\mathcal{I}_n = \mathcal{I}'_n$ 
23:   else  $\mathcal{I}_n = \mathcal{I}_{n-1}$ 
24:   if  $max < p(\mathcal{I}_n)$ 
25:      $\mathcal{I}_{max} = \mathcal{I}_n$ ,  $max = p(\mathcal{I}_n)$ 
26: end for
27: return  $\mathcal{I}_{max}$ .
```

An advantage of the above proposal distribution is that it is symmetric, e.g., $q(\mathcal{I}^*|\mathcal{I}) = q(\mathcal{I}|\mathcal{I}^*)$. Eq. 11 becomes:

$$\alpha(\mathcal{I} \rightarrow \mathcal{I}^*) = \min \left(1, \frac{p(\mathcal{I}^*)}{p(\mathcal{I})} \right). \quad (12)$$

The pseudocode of our Metropolis video synopsis algorithm is listed in Alg. 1.

In the following, we give more implementation details of the proposed method.

1) *Acceleration by GPU*: It usually takes 30 million iterations to obtain a good solution by the metropolis algorithm, which cost our un-optimized implementation about 5 minutes on an Intel(R) Core(TM) i7-6700HQ 2.60GHz CPU. The bottleneck is the calculation of Eq. 2. Note that Eq. 2 is composed of many terms which can be computed in parallel. We put the computation of these terms on the NVIDIA GeForce GTX 960M card. This accelerates our algorithm about 100 times to output a solution within several seconds.

2) *Refining Speed and Size Variables*: The solution produced by Alg. 1 is good but not perfect. We further enforce



Fig. 5. We use a status bar to visualize the speed of objects.

the smoothness of these variables by minimizing:

$$E_{refine}(\{\tilde{x}_i^j\}) = \sum_i \sum_{j=1}^{M_i} (\tilde{x}_i^j - x_i^j)^2 + \sum_i \sum_{j=2}^{M_i-1} (2\tilde{x}_i^j - \tilde{x}_i^{j-1} - \tilde{x}_i^{j+1})^2, \quad (13)$$

where x_i^j can be v_i^j or r_i^j . The first term of Eq. 13 is called data term that makes $\tilde{x}_i^j$ resemble x_i^j , while the second term is called smooth term which penalizes the second-order derivatives of $\tilde{x}_i^j$.

3) *Visualization of Speed Changes*: To remind the user about the degree of an object been accelerated or decelerated, we visualize the speed of object as shown in Fig. 5. Besides the time, we show a status bar on the objects. If accelerated, the color of the rectangle is yellow; otherwise the color is blue. The more full the bar, the more the object being accelerated or decelerated.

IV. EXPERIMENTS

We have conducted a variety of experiments to validate the effectiveness of the proposed method, which are described in detail in this section. **We have also provided a supplemental video containing the experimental results. Please refer to it for more information.**

A. Test Videos

We test our method and compare it with existing approaches on 6 surveillance videos (Fig. 10 to 12, and Fig. 13 to 15). Two videos were found on the Internet (Fig. 10 and 11), and one was captured by ourselves (Fig. 12). Another one video is from [4] (Fig. 13), and the last two are from Li et al. [5] (Fig. 14 and 15). Typical scenarios, such as crossroad, sidewalk, and road are took into consideration. The original length, number of objects, and the resolution of these videos are listed in Table I. Since the collision term of Eq. 2 is defined on intersection area between objects, we resize the videos to make the objects of different videos have similar spatial sizes. For this, we compute the average object size of each video, and then find a resizing ratio for each video accordingly. The last column of Table I gives the new resolution of each video after resizing.

TABLE I
THE ORIGINAL LENGTH (ORI. LEN.), NUMBER OF MOVING OBJECTS (N), ORIGINAL RESOLUTION (ORI. RES.), AND NORMALIZED RESOLUTION (NOR. RES.) OF TEST VIDEOS.

	Video ID	Ori. Len.	N	Ori. Res.	Nor. Res.
Fig. 10	1	5'10"	26	368×276	359×269
Fig. 11	2	10'26"	40	368×276	1714×1286
Fig. 12	3	5'36"	16	640×368	1051×604
Fig. 13	4	3'	18	480×384	480×384
Fig. 14	5	5'	14	368×276	297×223
Fig. 15	6	7'59"	19	480×360	316×237

B. Evaluation Metrics

We use the following four metrics for the quantitative evaluations and comparisons in this paper. Similar metrics were also adopted in previous works like [5] and [6]:

Compression Ratio (CR): The CR is computed as the length of the synopsis video divided by that of the input video. As an example, video 1 in Table I is originally 5'10". We compress it to obtain a 21" synopsis video. The CR is 0.068. We also write it as 1 : 0.068.

Collision Artifact (CA): We use CA to measure the total collision artifacts between all pairs of objects, as defined in Eq. 4.

Chronological Disorder Number (CDN): This is used to measure the degree to which the chronological orders between objects is destroyed. To be consistent with previous approaches like [5], we compute it as the number of pairs of objects whose chronological orders are inverted.

Total Energy (TE): We also use the value of the total energy defined in Eq. 2 to measure the effectiveness of our method.

C. Parameter Analysis and Settings

In this section, we discuss the parameters in our method. L_s is the length of synopsis video, controlling the metric of **CR**. Different L_s are used in the following different experiments, and will be discussed in their respective positions. n_iters determines the number of iterations of our Metropolis sampling algorithm, which is the larger the better and usually 3×10^7 is enough.

Besides the above two parameters, we have another three kinds of relatively important parameters. They are: ω_c , ω_t , ω_s and ω_d which are the weights of the terms of Eq. 2, $[a, b]$ and $[c, d]$ which are the change ranges of speed and size, and ℓ which is the length of a segment. We discuss them in detail in the following three subsections.

1) *Term weights*: The four weights ω_c , ω_t , ω_s and ω_d are used to balance the five terms of Eq. 2 (the weight of the first term is always 1). Since the numerical differences of these five energy terms are very large, the four weights may be also very different, and the differences may span several orders of magnitude. Therefore, it is not easy to test all combinations of the weights to find the best value for each of them. But we find that if we increase the weight of any term, the penalty defined by that term will decrease. This is verified by experiments in Fig. 6, in which we increase each of the four weights separately, and the corresponding energy term all decreases.

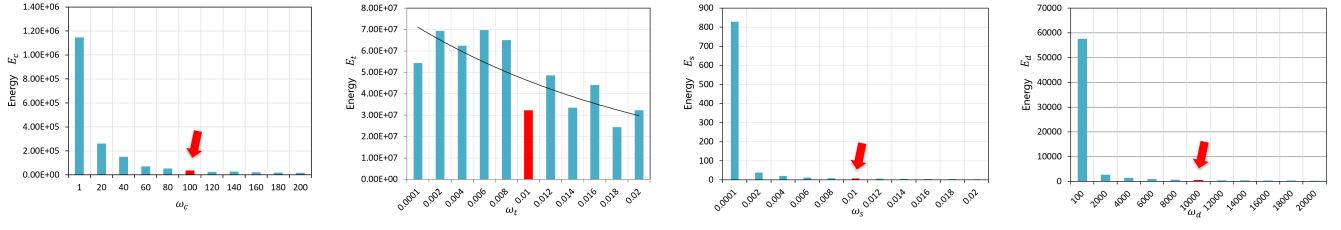


Fig. 6. The increase of any weight all leads to the decrease of the corresponding energy term. The experiments are conducted on the test video as shown in Fig. 10 without loss of generality. L_s is set as 21 seconds. When changing a weight, all other weights are kept as their default values.

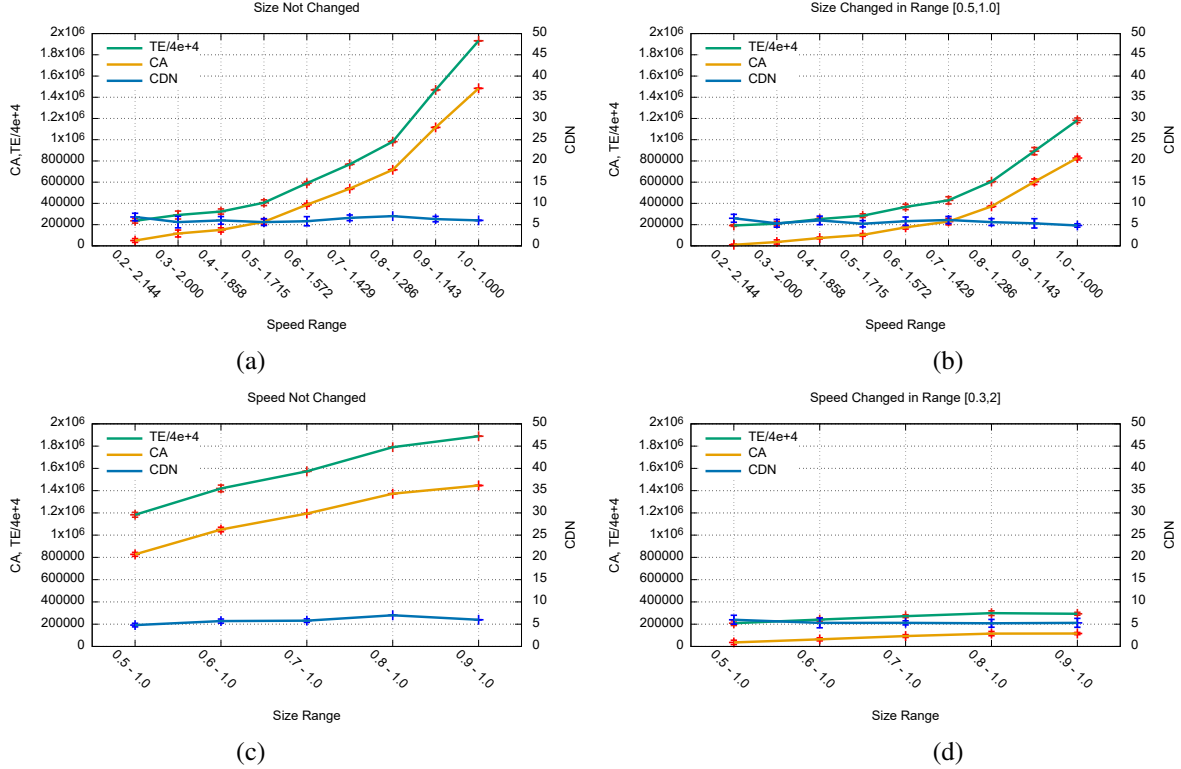


Fig. 7. (a) Size does not change, and then gradually reduce the speed range $[a, b]$ from $[0.2, 2.144]$ to $[1.0, 1.0]$. (b) Size is allowed to change in the range of $[0.5, 1.0]$, and then gradually reduce the speed range $[a, b]$ from $[0.2, 2.144]$ to $[1.0, 1.0]$. (c) Speed does not change, and then gradually reduce the size range $[c, d]$ from $[0.5, 1.0]$ to $[1.0, 1.0]$. (d) Speed is allowed to change in the range of $[0.3, 2.0]$, and then gradually reduce the size range $[c, d]$ from $[0.5, 1.0]$ to $[1.0, 1.0]$.

According to this observation, we can find good enough values for the four weights by adjusting them manually. For example, when we find there are too many collision artifacts in the synopsis result, we increase the weight ω_c by an order of magnitude. Or if the speed or size of objects does not change, we decrease the weight ω_d by an order of magnitude. After several trial-and-error, we find that $\omega_c = 100$, $\omega_t = 0.01$, $\omega_s = 0.01$, $\omega_d = 10^4$ work well for all the test videos, and set them as the default values for the four weights.

2) *Speed and size ranges*: Fig. 7 and Fig. 8 show the experimental results about speed and size ranges $[a, b]$ and $[c, d]$. Without loss of generality, the experiments are conducted on the test video as shown in Fig. 14, and L_s is set as 46 seconds.

In Fig. 7 (a), we do not allow object size to change, and then gradually reduce the speed range from $[0.2, 2.144]$ to $[1.0, 1.0]$. It can be seen that the metrics of **CR** and **TE** increase significantly. This means that the more we allow

speed to change, the more effective the collision artifacts can be avoided and better synopsis results can be obtained. The experiments in (b) are similar to those in (a), except that the size is now allowed to vary in the range of $[0.5, 1]$. In this case, **CR** and **TE** also increase but with a slower rate. The comparison between (a) and (b), i.e., the comparison between without or with size change, shows that allowing object size to change can further improve the effectiveness of our method.

In Fig. 7 (c), we do not allow object speed to change, and then gradually reduce the size range from $[0.5, 1.0]$ to $[1.0, 1.0]$. Not surprisingly, the metrics of **CR** and **TE** increase, indicating that allowing size to change can indeed help avoid collision artifacts and produce better synopsis results. But one observation demonstrates that changing speed is more effective than changing size to obtain better synopsis results. That is, the growth rates of the **CA** and **TE** curves in (a) are faster than those in (c). In Fig. 7 (d), we allow speed to change in the

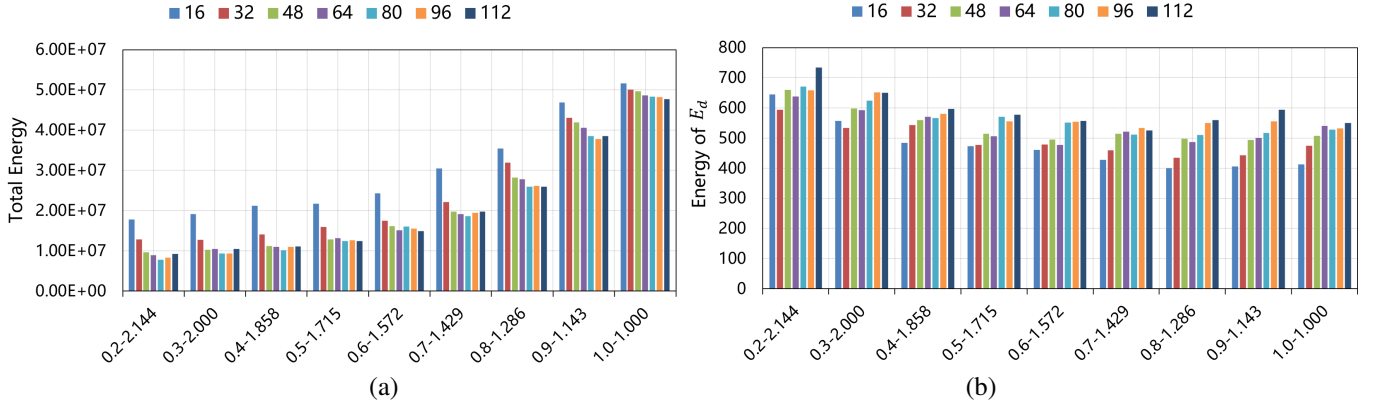


Fig. 9. (a) The change of the total energy with respect to change of speed range and ℓ . (b) The change of the energy of term E_d with respect to the change of speed range and ℓ .

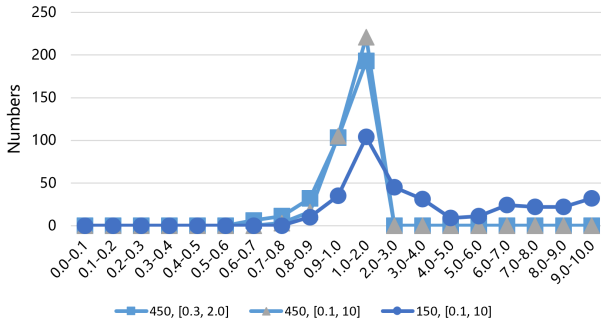


Fig. 8. “450, [0.3, 2.0]” means the input video is compressed to 450 frames, and the speed changes in the range of [0.3, 2.0]. For each of the settings, the distribution of the number of segments with respect to their speed is shown.

range of [0.3, 2.0], and then gradually reduce size range. In this case, the performance of our method does not change too much along with the change of the speed range. This is because the change of speed in the range of [0.3, 2.0] has already made our method able to produce good enough results, and the change of size is not necessary, which again demonstrates that the fact of speed is more important than size in our method.

The final point needed to be noted about Fig. 7 is that the metric of **CDN** does not change too much in all the experiments. This is due to the reason that the chronological term E_t is only related to shift variables rather than speed and size variables.

Fig. 7 suggests that the larger the ranges of speed and size, the better. In this paper, we set [0.3, 2.0] and [0.5, 1.0] as the default values for $[a, b]$ and $[c, d]$, respectively. We do this as we do not want to change the speed and size of objects too much. Otherwise, the activity details of objects will be lost severely. But the users are free to increase the speed and size ranges as they want. The larger the ranges, the better the collision and chronological disorder artifacts will be avoided, and larger compression ratio can be obtained. The only problem is that the objects will move more faster or become smaller.

Fig. 8 gives an example in which we compare between [0.3, 2.0] and [0.1, 10] for $[a, b]$. In the figure, the setting “450, [0.3, 2.0]” means the video is compressed to 450 frames, and

the speed changes in the range of [0.3, 2.0]. Then for each setting, we draw the curve which describes the distribution of the number of segments with respect to their speed. For setting “450, [0.3, 2.0]”, the speed of most segments is within [0.3, 2.0]. However, the distribution does not change too much when the setting becomes “450, [0.1, 10]”, even though the speed is now allowed to be increased or decreased more. This is because a 450-frame synopsis video is relatively long. It is enough for the objects to change their speed in [0.3, 2.0] to obtain a good result. However, things change when we compress the video to 150 frames. Now there are many segments whose speed is within [2.0, 10.0].

We provide a supplemental video that contains the synopsis results corresponding to the above settings. Please refer to it for more intuitive impression of how things change when speed range expands.

3) *Segment length*: Another important parameter is ℓ that determines the length of a segment. The value of ℓ should not be too large nor too small. Firstly, ℓ should satisfy $0 < a \leq 1 \leq b \leq \ell$. If ℓ is small, b should be small too, which means we can not set a large speed range. Smaller ℓ also brings more speed and size variables that enlarge the solution space, and more Metropolis sampling iterations will be required to find a good solution. On the contrary, the larger the ℓ , the fewer the segments, and there will be less flexibility for the objects to avoid collision and chronological disorder artifacts.

We use the experiments in Fig. 9 to determine the best value for ℓ . In (a), we show the relationship between the total energy and the variables of speed range and ℓ . In (b), the total energy is replaced by the energy of term E_d . The value of ℓ is selected among 16, 32, 48, 64, 80, 90, and 112, and the speed range is gradually reduced from [0.2, 2.144] to [1.0, 1.0]. Fig. 9 (a) suggests that ℓ is the larger the better as larger ℓ leads to smaller total energy. But Fig. 9 (b) indicates ℓ is the smaller the better as smaller ℓ usually yields smaller E_d which means object speed and size are less changed. Taking the two aspects into consideration, we choose 48 as the default value for ℓ .

Fig. 9 also shows that for different speed ranges, the changing trend of both total energy and the energy of E_d with respect to ℓ is very similar. This indicates that there is no relationship between parameters ℓ and speed range $[a, b]$.



Fig. 10. Input: 5'10". Output: 21". Compression ratio: 1 : 0.068. First row: results of SHIFT which contain lots of collision artifacts. Second row: results of SHIFT+SIZE which contain much fewer collisions but objects A and B, objects C and D still collide with each other as shown in (e) and (f). Last row: the results of our COMPLETE method which contain no collision artifact.

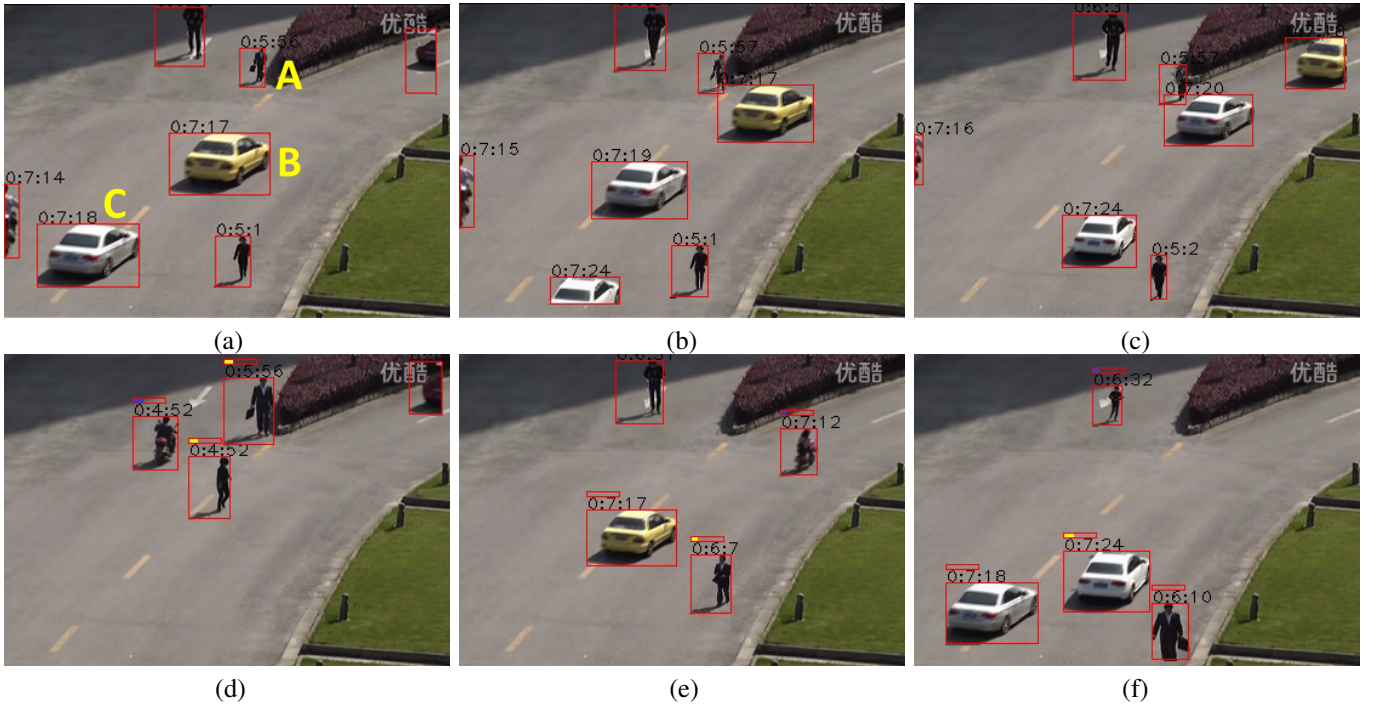


Fig. 11. Input: 10'26". Output: 1'20". Compression ratio: 1 : 0.128. First row: results of SHIFT+SIZE which contain fewer collisions but destroy the chronological orders much. Second row: results of COMPLETE in which both the collision and chronological requirements are satisfied.

D. Comparisons Among Variants of Our Method

In this section, we perform comparisons between our method and two variants of it.

One variant is to set both the speed and size ranges as [1, 1]. This variant only shifts objects and we call it SHIFT. The

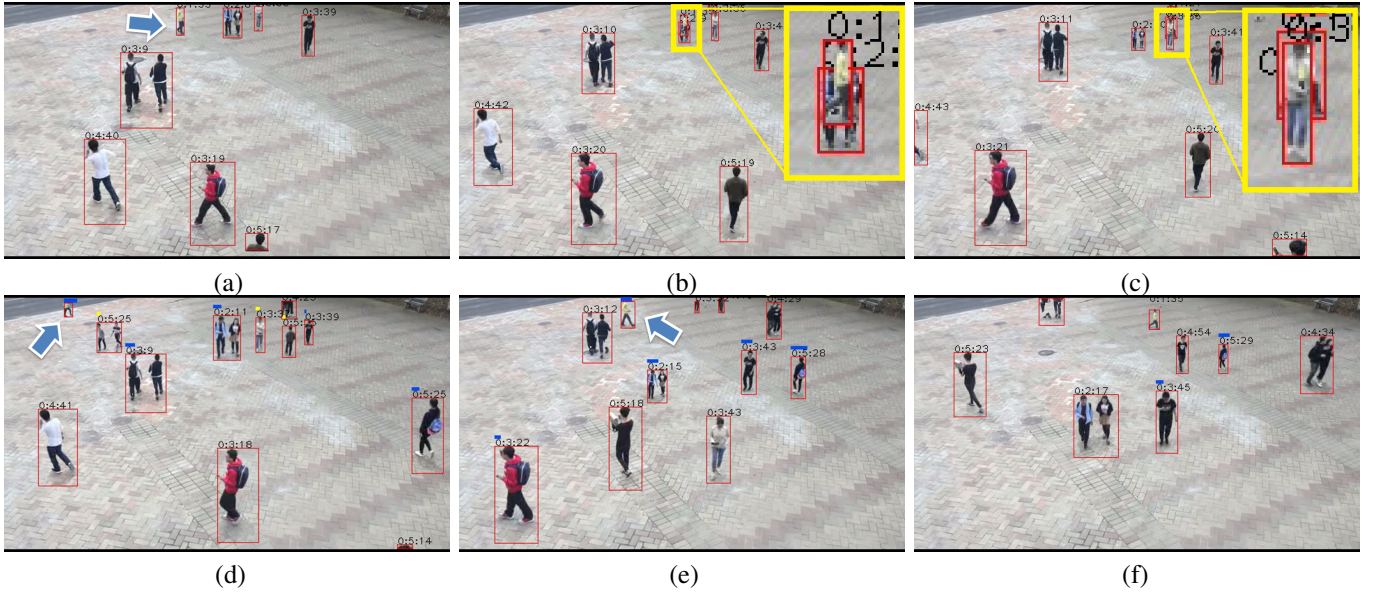


Fig. 12. Input: 5'36". Output: 25". Compression ratio: 1 : 0.074. First row: results of SHIFT+SIZE. Second row: results of COMPLETE. In (d) and (e), the object indicated by the arrow is decelerated to avoid collisions in (b) and (c).

other variant is to set the speed range as $[1, 1]$, and the size range as $[0.5, 1]$, which is called SHIFT+SIZE. Finally, the COMPLETE algorithm is the one that incorporates both speed and size changes.

Fig. 10, 11, and 12 present comparisons among SHIFT, SHIFT+SIZE, and COMPLETE. Please also refer to our supplemental video for more comparisons. Fig. 10 shows a sidewalk scene. The input video is 5 minutes and 10 seconds, and the synopsis video is 21 seconds. The compression ratio is 0.068. The first row shows results of SHIFT, the second row shows results of SHIFT+SIZE, and the last row shows results of COMPLETE. As can be seen, there are lots of collisions between objects in the first row. By scaling down the objects, the SHIFT+SIZE variant reduces the collision artifacts a lot. But as can be seen in the second row the objects A and B, and objects C and D still collide with each other. By additionally changing the speed of the objects, our COMPLETE method eliminates all the collision artifacts. From the frames in the last row, we can see that the objects A and D are accelerated (see the yellow rectangles on the top of them), while objects B and C are decelerated (see the blue rectangles on the top of them). Based on all of these accelerations and decelerations, the objects avoid the collisions smartly.

In Fig. 11 and 12, we only show the results of SHIFT+SIZE and COMPLETE. In Fig. 11, the SHIFT+SIZE method produces fewer collisions, but the chronological orders are destroyed much. For example, see the objects A, B, and C in Fig. 11 (a) which appear at 05 : 56, 07 : 17, and 07 : 18 respectively. By SHIFT+SIZE, objects B and C occur too earlier such that A collide with them sequentially in Fig. 11 (b) and (c). The COMPLETE method delays the occurring of B and C so that the collisions are avoided. The delaying also makes the chronological positions of the objects in (d)-(f) more natural than those in (a)-(c). Fig. 12 shows that the object indicated by the arrow in (d) and (e) is decelerated to

avoid the collisions in (b) and (c).

TABLE II
QUANTITATIVE COMPARISONS BETWEEN NIE ET AL. [4], LI ET AL. [5] AND OUR COMPLETE METHOD. **COLLISION COUNT** IS THE NUMBER OF COLLISIONS THAT OCCUR IN A SYNOPSIS VIDEO.

	Fig. 13		Fig. 14		Fig. 15	
	[4]	Our	[5]	Our	[5]	Our
CR	0.083	0.083	0.153	0.153	0.136	0.136
CDN	8	4	5	0	9	0
Collision Count	0	0	1	0	4	0

E. Comparisons with Previous Approaches

In this section, we compare our method COMPLETE with approaches proposed by Nie et al. [4] that shifts objects in both spatial and temporal domains and Li et al. [5] that resizes objects down. The quantitative experimental results are shown in Table II. Fig. 13, 14 and 15 show frames that are from the synopsis videos generated by the three methods. Please refer to the supplemental video that contains these synopsis results for more information.

In Fig. 13, we compare our method with the one proposed by Nie et al. [4]. The first row shows results of [4] and the second row presents our results. The compression ratio is 0.083 which is very high. As can be seen from Table II, both [4] and our method avoid collision artifacts successfully. But for the method of [4] the chronological disorder number (CDN) is 8, while it is only 4 for our method. Another problem of [4] is that by shifting objects along the x and y axes, it severely destroys spatial relationships between objects. Furthermore, it needs to change the background of the original scene to match with the spatially-shifted objects which however is sometimes very confusing. In this sense, our method produces more natural synopsis results compared to [4].

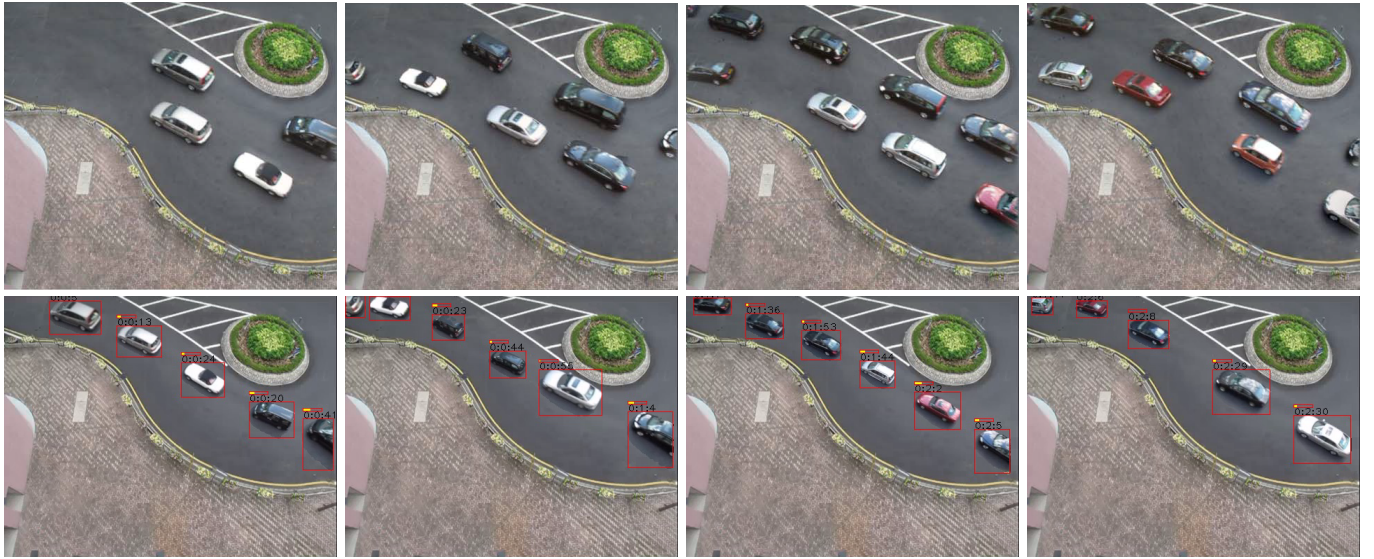


Fig. 13. Comparison with [4]. Input: 3'. Output: 15''. Compression ratio: 1 : 0.083. First row: results of [4]. Second row: results of COMPLETE.

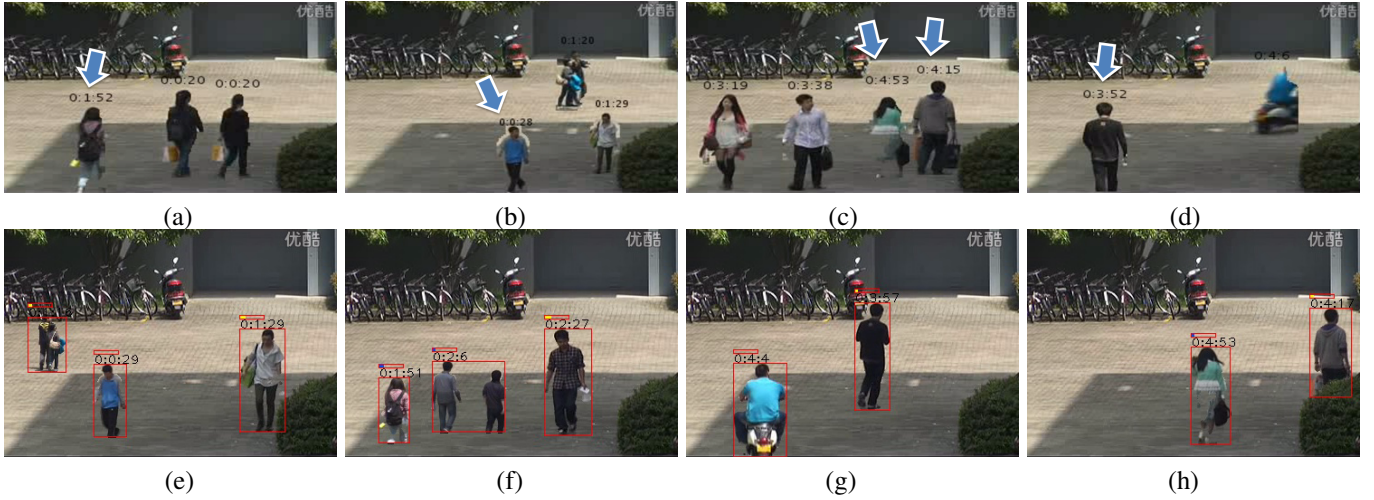


Fig. 14. Comparison with [5]. Input: 5'. Output: 46''. Compression ratio: 1 : 0.153. First row: results of [5]. Second row: results of COMPLETE.

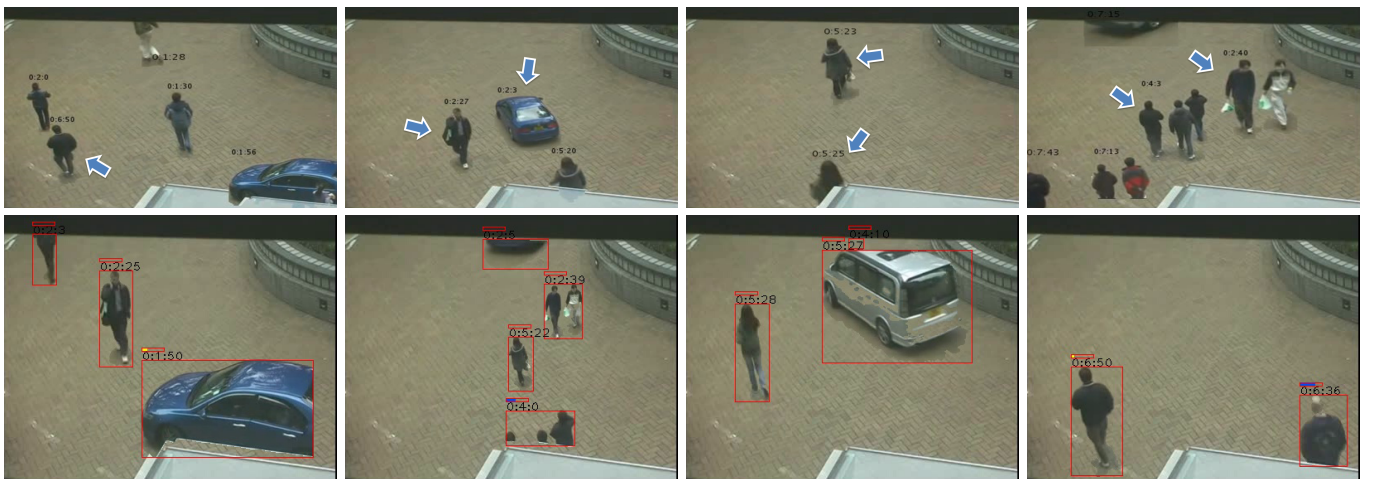


Fig. 15. Comparison with [5]. Input: 7'59''. Output: 1'05''. Compression ratio: 1:0.136. First row: results of [5]. Second row: results of COMPLETE.

In Fig. 14 and Fig. 15, we compare our method with [5]. In each figure, the first row shows results of [5], and the second row shows results of our COMPLETE. For the example in Fig. 14 the compression ratio (**CR**) is 0.153, while for the second example in Fig. 15 the **CR** is 0.136. As can be seen from Table II, in both examples the method of [5] produces many artifacts in terms of **CDN** and **Collision Count**, while our method avoids these artifacts completely, which demonstrate that our method performs better than [5]. The arrows in the first row of Fig. 14 and Fig. 15 indicate some examples of chronological disorders. As an example, the 0:1:52 object appears in Fig. 14 (a), but the 0:0:28 object appears later in Fig. 14 (b), which is not natural.

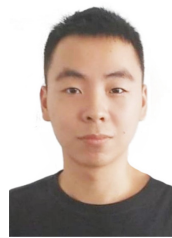
V. CONCLUSION AND FUTURE WORK

In this paper, we incorporate both object speed and size changes into the technique of object shifting based video synopsis and achieve good synopsis results with no collision artifact and very high compression ratio. By object shifting, we compress the objects from the long input video into a short synopsis video. By speed and size changing, the objects become very flexible to avoid collision and chronological disorder artifacts. We have designed a unified optimization framework that performs object shifting, speed changing, and size scaling simultaneously, and also proposed a Metropolis sampling algorithm to find the solution for the optimization. We have conducted a variety of experiments to demonstrate that the proposed method can produce better synopsis results than previous approaches. A limitation of our method is that it treats two objects as a single one if they originally collide with each other in the input video. This harms the compression ratio. We will try to solve this problem in the future.

REFERENCES

- [1] A. Rav-Acha, Y. Pritch, and S. Peleg, "Making a long video short: Dynamic video synopsis," in *Computer Vision and Pattern Recognition, 2006 IEEE Computer Society Conference on*, vol. 1. IEEE, 2006, pp. 435–441.
- [2] Y. Pritch, A. Rav-Acha, A. Gutman, and S. Peleg, "Webcam synopsis: Peeking around the world," in *Computer Vision, 2007. ICCV 2007. IEEE 11th International Conference on*. IEEE, 2007, pp. 1–8.
- [3] Y. Pritch, A. Rav-Acha, and S. Peleg, "Nonchronological video synopsis and indexing," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 11, pp. 1971–1984, 2008.
- [4] Y. Nie, C. Xiao, H. Sun, and P. Li, "Compact video synopsis via global spatiotemporal optimization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 19, no. 10, pp. 1664–1676, 2013.
- [5] X. Li, Z. Wang, and X. Lu, "Surveillance video synopsis via scaling down objects," *IEEE Transactions on Image Processing*, vol. 25, no. 2, pp. 740–755, 2016.
- [6] Y. He, C. Gao, N. Sang, Z. Qu, and J. Han, "Graph coloring based surveillance video synopsis," *Neurocomputing*, vol. 225, pp. 64–79, 2017.
- [7] B. T. Truong and S. Venkatesh, "Video abstraction: A systematic review and classification," *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, vol. 3, no. 1, p. 3, 2007.
- [8] T. Liu, X. Zhang, J. Feng, and K.-T. Lo, "Shot reconstruction degree: a novel criterion for key frame selection," *Pattern recognition letters*, vol. 25, no. 12, pp. 1451–1457, 2004.
- [9] G. Ciocca and R. Schettini, "An innovative algorithm for key frame extraction in video summarization," *Journal of Real-Time Image Processing*, vol. 1, no. 1, pp. 69–88, Mar 2006. [Online]. Available: <https://doi.org/10.1007/s11554-006-0001-1>
- [10] Y. Zhang, R. Tao, and Y. Wang, "Motion-state-adaptive video summarization via spatiotemporal analysis," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 27, no. 6, pp. 1340–1352, 2017.
- [11] K. Kumar, D. D. Shrimankar, and N. Singh, "Eratosthenes sieve based key-frame extraction technique for event summarization in videos," *Multimedia Tools and Applications*, vol. 77, no. 6, pp. 7383–7404, 2018.
- [12] C. M. Taskiran, Z. Pizlo, A. Amir, D. Ponceleon, and E. J. Delp, "Automated video program summarization using speech transcripts," *IEEE Transactions on Multimedia*, vol. 8, no. 4, pp. 775–791, 2006.
- [13] X. Zhu, C. C. Loy, and S. Gong, "Learning from multiple sources for video summarisation," *International Journal of Computer Vision*, vol. 117, no. 3, pp. 247–268, 2016.
- [14] M. Paul and M. M. Salehin, "Spatial and motion saliency prediction method using eye tracker data for video summarization," *IEEE Transactions on Circuits and Systems for Video Technology*, 2018.
- [15] K.-Y. Cheng, S.-J. Luo, B.-Y. Chen, and H.-H. Chu, "Smartplayer: user-centric video fast-forwarding," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 2009, pp. 789–798.
- [16] H. Huang, D. Lischinski, Z. Hao, M. Gong, M. Christie, and D. Cohen-Or, "Trip synopsis: 60km in 60sec," in *Computer Graphics Forum*, vol. 35, no. 7. Wiley Online Library, 2016, pp. 107–116.
- [17] S. Lan, R. Panda, Q. Zhu, and A. K. Roy-Chowdhury, "Ffnet: Video fast-forwarding via reinforcement learning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 6771–6780.
- [18] E. P. Bennett and L. McMillan, "Computational time-lapse video," in *ACM Transactions on Graphics (TOG)*, vol. 26, no. 3. ACM, 2007, p. 102.
- [19] M. Wang, J.-B. Liang, S.-H. Zhang, S.-P. Lu, A. Shamir, and S.-M. Hu, "Hyper-lapse from multiple spatially-overlapping videos," *IEEE Transactions on Image Processing*, vol. 27, no. 4, pp. 1735–1747, 2018.
- [20] V. Valdés and J. M. Martínez, "On-line video skimming based on histogram similarity," in *Proceedings of the international workshop on TRECVID video summarization*. ACM, 2007, pp. 94–98.
- [21] Z. Longfei, C. Yuanda, D. Gangyi, and W. Yong, "A computable visual attention model for video skimming," in *Multimedia, 2008. ISM 2008. Tenth IEEE International Symposium on*. IEEE, 2008, pp. 667–672.
- [22] A. G. del Molino, X. Boix, J.-H. Lim, and A.-H. Tan, "Active video summarization: Customized summaries via on-line interaction with the user," in *AAAI*, 2017, pp. 4046–4052.
- [23] X. Li, B. Zhao, and X. Lu, "A general framework for edited video and raw video summarization," *IEEE Transactions on Image Processing*, vol. 26, no. 8, pp. 3652–3664, 2017.
- [24] B. Zhao, X. Li, and X. Lu, "Hsa-rnn: Hierarchical structure-adaptive rnn for video summarization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7405–7414.
- [25] H.-W. Kang and X.-Q. Chen, "Space-time video montage," in *computer vision and pattern recognition, 2006 IEEE computer society conference on*, vol. 2. IEEE, 2006, pp. 1331–1338.
- [26] C. D. Correa and K.-L. Ma, "Dynamic video narratives," in *ACM Transactions on Graphics (TOG)*, vol. 29, no. 4. ACM, 2010, p. 88.
- [27] C. Barnes, D. B. Goldman, E. Shechtman, and A. Finkelstein, "Video tapestries with continuous temporal zoom," in *ACM Transactions on Graphics (TOG)*, vol. 29, no. 4. ACM, 2010, p. 89.
- [28] T. Chen, A. Lu, and S.-M. Hu, "Visual storylines: Semantic visualization of movie sequence," *Computers & Graphics*, vol. 36, no. 4, pp. 241–249, 2012.
- [29] P. Dragicevic, G. Ramos, J. Bibliowicz, D. Nowrouzezahrai, R. Balakrishnan, and K. Singh, "Video browsing by direct manipulation," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 2008, pp. 237–246.
- [30] T. Karrer, M. Weiss, E. Lee, and J. Borchers, "Dragon: a direct manipulation interface for frame-accurate in-scene video navigation," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 2008, pp. 247–250.
- [31] S.-P. Lu, S.-H. Zhang, J. Wei, S.-M. Hu, and R. R. Martin, "Timeline editing of objects in video," *IEEE Transactions on Visualization and Computer Graphics*, vol. 19, no. 7, pp. 1218–1227, 2013.
- [32] Y. Pritch, S. Ratovitch, A. Hendel, and S. Peleg, "Clustered synopsis of surveillance video," in *2009 Advanced Video and Signal Based Surveillance*. IEEE, 2009, pp. 195–200.
- [33] M. Rodriguez, "Cram: Compact representation of actions in movies," in *Computer Vision and Pattern Recognition (CVPR), 2010 IEEE Conference on*. IEEE, 2010, pp. 3328–3335.
- [34] W. Lin, Y. Zhang, J. Lu, B. Zhou, J. Wang, and Y. Zhou, "Summarizing surveillance videos with local-patch-learning-based abnormality detection, blob sequence optimization, and type-based synopsis," *Neurocomputing*, vol. 155, pp. 84–98, 2015.

- [35] W.-C. Wang, P.-C. Chung, C.-R. Huang, and W.-Y. Huang, "Event based surveillance video synopsis using trajectory kinematics descriptors," in *Machine Vision Applications (MVA), 2017 Fifteenth IAPR International Conference on*. IEEE, 2017, pp. 250–253.
- [36] M. Lu, Y. Wang, and G. Pan, "Generating fluent tubes in video synopsis," in *Acoustics, Speech and Signal Processing (ICASSP), 2013 IEEE International Conference on*. IEEE, 2013, pp. 2292–2296.
- [37] X. Li, Z. Wang, and X. Lu, "Video synopsis in complex situations," *IEEE Transactions on Image Processing*, vol. 27, no. 8, pp. 3798–3812, 2018.
- [38] S. Feng, Z. Lei, D. Yi, and S. Z. Li, "Online content-aware video condensation," in *Computer Vision and Pattern Recognition (CVPR), 2012 IEEE Conference on*. IEEE, 2012, pp. 2082–2087.
- [39] C.-R. Huang, P.-C. J. Chung, D.-K. Yang, H.-C. Chen, and G.-J. Huang, "Maximum a posteriori probability estimation for online surveillance video synopsis," *IEEE Transactions on circuits and systems for video technology*, vol. 24, no. 8, pp. 1417–1429, 2014.
- [40] J. Zhu, S. Feng, D. Yi, S. Liao, Z. Lei, and S. Z. Li, "High-performance video condensation system," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 25, no. 7, pp. 1113–1124, 2015.
- [41] M. Ra and W.-Y. Kim, "Parallelized tube rearrangement algorithm for online video synopsis," *IEEE Signal Processing Letters*, vol. 25, no. 8, 2018.
- [42] Z. Li, P. Ishwar, and J. Konrad, "Video condensation by ribbon carving," *IEEE Transactions on Image Processing*, vol. 18, no. 11, pp. 2572–2583, 2009.
- [43] Y.-M. Liang, "Video condensation for video forensics," in *Computational Intelligence and Cybernetics (CyberneticsCom), 2012 IEEE International Conference on*. IEEE, 2012, pp. 180–184.
- [44] H. T. Nguyen, S.-W. Jung, and C. S. Won, "Order-preserving condensation of moving objects in surveillance videos," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 9, pp. 2408–2418, 2016.
- [45] Y. Nie, H. Sun, P. Li, C. Xiao, and K.-L. Ma, "Object movements synopsis via part assembling and stitching," *IEEE transactions on visualization and computer graphics*, vol. 20, no. 9, pp. 1303–1315, 2014.
- [46] Z. Zivkovic and F. Van Der Heijden, "Efficient adaptive density estimation per image pixel for the task of background subtraction," *Pattern recognition letters*, vol. 27, no. 7, pp. 773–780, 2006.
- [47] Q. Wang, L. Zhang, L. Bertinetto, W. Hu, and P. H. Torr, "Fast online object tracking and segmentation: A unifying approach," *arXiv preprint arXiv:1812.05050*, 2018.
- [48] V. Kolmogorov and R. Zabini, "What energy functions can be minimized via graph cuts?" *IEEE transactions on pattern analysis and machine intelligence*, vol. 26, no. 2, pp. 147–159, 2004.
- [49] M. Lourakis, "levmar: Levenberg-marquardt nonlinear least squares algorithms in C/C++," [web page] <http://www.ics.forth.gr/~lourakis/levmar/>, Jul. 2004, [Accessed on 31 Jan. 2005].
- [50] W. R. Gilks, S. Richardson, and D. Spiegelhalter, *Markov chain Monte Carlo in practice*. CRC press, 1995.
- [51] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *The journal of chemical physics*, vol. 21, no. 6, pp. 1087–1092, 1953.
- [52] W. K. Hastings, "Monte carlo sampling methods using markov chains and their applications," 1970.



Zhenkai Li is now studying at the School of Computer Science & Engineering, South China University of Technology. His research interests fall in image and video processing.



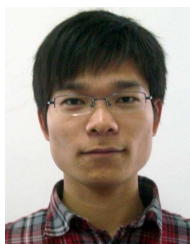
Zhensong Zhang is pursuing his PhD degree in the Department of Computer Science and Engineering, The Chinese University of Hong Kong. He received the B.E. degree from Xidian University in 2011 and the MS degree from University of Chinese Academy of Sciences in 2014. His research interests include image and video editing, and computational photography.



Qing Zhang received the B.Sc. and Ph.D. degrees in Computer Science and Engineering from Wuhan University in 2011 and 2017, respectively. He is currently a research associate professor at the School of Data and Computer Science, Sun Yat-Sen University. His research interest includes computer graphics and computer vision.



Tiezheng Ma is now studying at the School of Computer Science & Engineering, South China University of Technology. His research interests fall in image and video processing.



Yongwei Nie received the BSc and PhD degrees from the Computer School of Wuhan University in 2009 and 2015, respectively. Currently, he is an associate researcher at the School of Computer Science & Engineering, South China University of Technology. His research interests include image and video editing, and computational photography.



Hanqiu Sun is the visiting professor at Shanghai Jiao-tong University and Zhejiang University, adjunct professor at University of ESTC. Her current research interests include virtual reality, interactive graphics, real-time hypermedia, virtual surgery, effective image/video synopsis, panoramic video displays, dynamics and touch-enhanced simulations. Sun has an MS in electrical engineering from University of British Columbia, and PhD in computer science from University of Alberta, Canada. Contact her at sunnyqiu24@gmail.com.